AK Series Motors — Compatibility & Adapter Guide

Using different AK motors and switching CAN adapters

---

1. Using Different AK Series Motors

Will it work?

Yes. The entire CubeMars AK series — AK10-9, AK45-10, AK45-36, AK60-6, AK70-10, AK80-9 and others — all use the same CAN communication protocol and the same servo mode frame format. The cubemars_hardware package works with any of them. The only difference between motors in the config is four parameters in the URDF that describe the motor's physical characteristics. Everything else — the bridge, the controller setup, how you send commands — stays identical.

The Four Parameters That Change Per Motor

These come from the motor's datasheet, which you can find on the CubeMars website for each model:

```
<joint name="your_joint_name">
    <param name="can_id">111</param>      <!-- unique ID, set via R-Link, must be
                                               different for every motor on the bus -->
    <param name="pole_pairs">14</param>   <!-- from datasheet: number of pole pairs
                                               in the motor stator -->
    <param name="gear_ratio">36</param>   <!-- from datasheet: planetary gearbox
                                               reduction ratio -->
    <param name="kt">0.11</param>         <!-- from datasheet: torque constant in
                                               Nm/A, used to convert current to torque -->

    <command_interface name="position"/>
    <state_interface name="position"/>
    <state_interface name="velocity"/>
    <state_interface name="effort"/>
</joint>
```

What each parameter does

can_id — this is the address the motor listens on. Every motor on the CAN bus must have a unique ID. You set this using the R-Link and Upper Computer software before connecting to ROS 2. The package uses this to filter incoming status frames and address outgoing commands.

pole_pairs — brushless motors work by energising pairs of magnetic poles. The motor spins in electrical RPM, which is pole_pairs times the mechanical RPM. The package uses this to convert the raw electrical speed from the CAN frame into real radians per second.

gear_ratio — the planetary gearbox reduces the motor's output speed and multiplies its torque. Combined with pole_pairs, this converts the motor's internal encoder position into the actual output shaft angle in radians.

kt — the torque constant relates current to torque (torque = kt * current). The package uses this to convert the raw current reading from the CAN frame into the effort state interface value in Newton-metres.

Reference Parameters for Common AK Motors

These values come from CubeMars datasheets. Always verify against the datasheet for your specific motor version:

Motor	Pole Pairs	Gear Ratio	Kt (Nm/A)	Peak Torque
AK45-36 KV80	14	36	0.11	24 Nm
AK45-10 KV75	14	10	0.11	7 Nm
AK70-10 KV100	21	10	0.095	24.8 Nm
AK80-9 V3.0 KV100	21	9	0.091	22 Nm
AK60-6 V3.0 KV80	21	6	0.068	9 Nm
AK10-9 V3.0 KV60	21	9	0.16	53 Nm

> Note: pole pairs for AK70-10 and AK80-9 may vary between hardware versions — always cross-check with your specific motor's datasheet on cubemars.com.

Example: Mixed Motor Setup (AK45-36 + AK70-10)

If you were running an AK45-36 and an AK70-10 on the same bus, the URDF ros2_control section would look like this:

```
<ros2_control name="arm_system" type="system">
  <hardware>
    <plugin>cubemars_hardware/CubeMarsSystemHardware</plugin>
    <param name="can_interface">vcan0</param>
  </hardware>

  <joint name="shoulder_joint">   <!-- AK70-10 -->
    <param name="can_id">111</param>
    <param name="pole_pairs">21</param>
    <param name="gear_ratio">10</param>
    <param name="kt">0.095</param>
    <command_interface name="position"/>
    <state_interface name="position"/>
    <state_interface name="velocity"/>
    <state_interface name="effort"/>
  </joint>

  <joint name="wrist_joint">      <!-- AK45-36 -->
    <param name="can_id">112</param>
    <param name="pole_pairs">14</param>
    <param name="gear_ratio">36</param>
    <param name="kt">0.11</param>
    <command_interface name="position"/>
    <state_interface name="position"/>
    <state_interface name="velocity"/>
    <state_interface name="effort"/>
  </joint>
</ros2_control>
```

---

2. Switching to a Native SocketCAN Adapter

Why the Waveshare adapter needed a bridge

The Waveshare USB-CAN-A uses a proprietary serial protocol — every CAN frame is wrapped in 0xAA...0x55 bytes and sent over a virtual COM port. Linux's standard SocketCAN system doesn't understand this format, so we wrote waveshare_can_bridge.py to translate between the two.

A native SocketCAN adapter eliminates this entirely. The adapter presents itself directly as a CAN network interface (e.g. can0) and the kernel handles all the low-level framing. No bridge script needed, lower latency, and more robust at high update rates.

Recommended Native Adapters

CANable Pro — affordable USB adapter, works natively with SocketCAN using candleLight firmware. Widely used in robotics. Around $30-40.

PEAK PCAN-USB — industrial grade, excellent Linux driver support, very reliable at 1 Mbit/s. More expensive but very solid.

Kvaser Leaf Light — another solid industrial option with good Linux support.

> Note: if buying a CANable, make sure it comes with or can be flashed with candleLight firmware, not SLCAN firmware — candleLight gives you a native SocketCAN interface which is simpler and faster.

What Changes in the Code

Switching adapter requires only two changes:

Change 1 — URDF parameter

In your urdf/ak45.urdf.xacro, change vcan0 to can0 (or whatever your adapter's interface name is):

```
<!-- Before (Waveshare bridge) -->
<param name="can_interface">vcan0</param>

<!-- After (native adapter) -->
<param name="can_interface">can0</param>
```

Change 2 — start.sh

Replace the bridge and vcan0 setup with the two commands to bring up the real CAN interface:

```
#!/bin/bash
set -e
SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"

# Set up real CAN interface (replaces vcan0 + bridge)
sudo ip link set can0 type can bitrate 1000000
sudo ip link set can0 up

echo "[mars_ws] Sourcing ROS 2..."
source /opt/ros/humble/setup.bash
source "$SCRIPT_DIR/install/setup.bash"

echo "[mars_ws] Launching..."
ros2 launch marsworks_ak45 ak45.launch.py

# Cleanup
sudo ip link set can0 down
```

What Does NOT Change

Everything else stays exactly the same:

```
- The C++ patches to cubemars_hardware (filter removal, MSG_DONTWAIT fix,
  ID masking) are still needed and still apply
- controllers.yaml is unchanged
- How you send trajectory commands is unchanged
```

```
- How you read joint states is unchanged
- Multi-motor daisy chain setup is unchanged
- The motor parameters in the URDF are unchanged
```

Verifying the Native Interface is Working

After bringing up can0, verify it's recognised and frames are flowing before launching ROS 2:

```
# Check interface is up
ip link show can0

# Should show something like:
# can0: <NOARP,UP,LOWER_UP> mtu 16 ...

# Check frames are arriving from the motor
# (motor must be powered and send_can_status enabled)
candump can0

# You should see frames like:
# can0  0000296F   [8]  01 9F 00 00 00 00 2A 00
```

> If candump shows frames, the hardware is working and you can launch the ROS 2 stack. The motor's status frame ID will always be (0x29 << 8) | motor_id regardless of which adapter you use.

---

3. Quick Reference

Finding motor parameters on the CubeMars website

Go to cubemars.com, find your motor model, and look for these fields in the specification table:

```
Pole Pairs  -> use directly as pole_pairs
Reduction Ratio -> use the number before :1 as gear_ratio
KT (Nm/A)   -> use directly as kt
CAN ID      -> set this yourself via R-Link, must be unique per motor
```

CAN bus termination reminder

The CAN bus requires a 120 ohm termination resistor at each physical end of the cable. With one motor: adapter terminates one end, motor terminates the other. With multiple motors daisy chained: adapter at one end, last motor in the chain at the other. Middle motors should have termination disabled. Check the AK series driver board for a termination jumper or solder bridge.

Motor CAN ID assignment

IDs are set using the R-Link V2.0 and the CubeMars Upper Computer software. Connect the motor via the UART port on the driver board, open the Upper Computer, go to Application Functions, and change the CAN ID there. IDs must be unique across all motors on the same bus. Recommended: use sequential IDs starting from 111 (111, 112, 113...) to make debugging easier.